PLANTILLA · SISTEMA OPERATIVO DE UN AGENTE DE IA

La estructura de 4 archivos

para que tu agente de IA no empiece de cero cada sesión

Agents.md · Context.md · Memory.md · Skills.md

Ejemplo aplicado a un proyecto real: SwissRoute, un backend en Java / Spring Boot para planificación de trayectos de transporte público en Suiza.

Preparado por Chantal — contenido educativo sobre agentes de IA

CÓMO USAR ESTA PLANTILLA

Antes de empezar

Un agente puede ser muy potente, pero sin una estructura clara, cada sesión empieza prácticamente desde cero. Esta plantilla organiza la información que necesita tu agente en cuatro archivos, cada uno con una función distinta. No son documentos aislados: juntos forman el sistema operativo del agente.

Cada sección de este PDF incluye la explicación del archivo y un ejemplo completo, aplicado a un proyecto real, listo para adaptar al tuyo.

---

01 `Agents.md`

Cómo debe comportarse el agente

02 `Context.md`

El proyecto: arquitectura y decisiones técnicas

03 `Memory.md`

Lo que no quieres volver a explicar

04 `Skills.md`

Las capacidades reutilizables del agente

05 `Equivalencias`

Cómo se llaman estos archivos en otras herramientas

01 · COMPORTAMIENTO

Agents.md

Define cómo debe comportarse el agente dentro de este proyecto.

Para qué sirve

Establece los límites de autonomía del agente: qué puede hacer sin preguntar, qué necesita validar antes de actuar, qué comandos debe usar y qué reglas nunca debe romper. Es el archivo que evita que el agente tome decisiones que no le corresponden.

Qué documentar aquí

- Acciones permitidas sin confirmación previa
- Acciones que siempre requieren validación humana
- Comandos exactos que debe usar (build, test, run)
- Reglas técnicas que nunca debe romper
- Formato esperado de commits y pull requests

Ejemplo real — proyecto SwissRoute

```
# Agents.md — SwissRoute

## Puede hacer sin preguntar
- Escribir y correr tests unitarios (JUnit + Mockito)
- Corregir errores de compilación y warnings del linter
- Actualizar comentarios Javadoc y el README
- Ejecutar `mvn test` y `mvn clean install`

## Debe validar antes de actuar
- Cualquier cambio en el esquema de la base de datos (migraciones Flyway)
- Cambios en la forma/firma de un endpoint público de la API
- Añadir o actualizar dependencias en pom.xml
- Cambios en la configuración de seguridad JWT

## Comandos del proyecto
- Build: mvn clean install
- Tests: mvn test
- Run local: mvn spring-boot:run
- Migraciones: mvn flyway:migrate

## Reglas que nunca debe romper
- Usar WebClient exclusivamente — RestTemplate está prohibido
- Toda llamada WebClient debe incluir manejo de errores con .onStatus()
- Nunca exponer entidades JPA directamente: toda respuesta pasa por un DTO
- Nunca loguear tokens JWT ni credenciales, ni en debug
- Los commits siguen Conventional Commits (feat:, fix:, refactor:, docs:)
```

Nota

Sé literal con los comandos — el agente los ejecuta tal cual, sin interpretarlos.

02 · CONTEXTO

Context.md

Explica el proyecto: qué es, cómo está construido y por qué.

Para qué sirve

Aquí documentas la arquitectura, los módulos principales, las dependencias, la estructura de carpetas y las decisiones técnicas importantes. Es el mapa que le permite al agente entender el proyecto sin que se lo tengas que explicar cada vez.

Qué documentar aquí

- Propósito del proyecto en una o dos frases
- Stack tecnológico completo
- Arquitectura general (capas, patrones usados)
- Estructura de carpetas con el propósito de cada una
- Integraciones externas y APIs de terceros
- Decisiones técnicas ya tomadas y por qué

Ejemplo real — proyecto SwissRoute

```
# Context.md — SwissRoute

## Propósito
Backend para planificar y consultar trayectos de transporte
público en Suiza. Proyecto de evaluación técnica, con alcance
dividido en dos sprints de una semana cada uno.

## Stack
- Java 17 + Spring Boot 3
- PostgreSQL
- WebClient (cliente HTTP reactivo)
- JWT para autenticación
- Swagger / OpenAPI para documentación (Sprint 2)

## Arquitectura
Capas: Controller -> Service -> Repository -> Entity
Los DTOs viven en su propio paquete y son obligatorios
en toda respuesta expuesta por la API.

## Integración externa
Swiss Public Transport API (transport.opendata.ch)
Endpoints en uso: /locations, /connections, /stationboard
Todas las llamadas pasan por un WebClient centralizado
con manejo de errores via .onStatus().

## Estructura de carpetas
/controller endpoints REST
/service lógica de negocio
/repository acceso a datos (JPA)
/dto objetos de entrada/salida de la API
/config beans (WebClient, seguridad JWT)
/client wrapper del API externo de transporte

## Decisiones técnicas
- WebClient sobre RestTemplate: soporte reactivo y mejor
manejo de timeouts frente al API externo
- JWT sobre sesiones: el proyecto no mantiene estado en servidor
```

Nota

Actualízalo cuando cambie la arquitectura, no cuando cambie el detalle de una función.

03 · MEMORIA

Memory.md

Guarda lo que no quieres volver a explicar.

Para qué sirve

Decisiones ya tomadas, errores que ya se solucionaron, preferencias de trabajo y restricciones que siguen siendo importantes en futuras sesiones. Este archivo solo debe conservar información estable y útil — no es un registro de todo lo que pasó.

Qué documentar aquí

- Decisiones ya cerradas (para no reabrir el debate)
- Errores ya resueltos y su causa raíz
- Preferencias de trabajo del desarrollador
- Restricciones activas del proyecto o del cliente
- Estado actual del plan (qué sprint, qué falta)

Ejemplo real — proyecto SwissRoute

```
# Memory.md — SwissRoute

## Decisiones cerradas — no reabrir
- WebClient es el único cliente HTTP permitido (decisión final,
ya evaluado RestTemplate y descartado)
- Autenticación JWT, sin sesiones de servidor

## Errores ya resueltos
- El API de transporte devuelve horarios en hora local suiza:
se normaliza a UTC en el DTO antes de guardar en BD
(bug detectado en HU-05, corregido convirtiendo con ZoneId
'Europe/Zurich' en el mapper de stationboard)
- Rate limiting del API externo: se agregó reintento con
backoff exponencial en el WebClient bean tras fallos 429

## Restricciones vigentes
- El API externo tiene límite de requests — evitar llamadas
innecesarias o en bucle durante pruebas
- El proyecto es de evaluación: no añadir features fuera de
las 14 user stories definidas sin confirmarlo antes

## Estado actual
- Sprint 1 (HU-01 a HU-07): setup, WebClient, registro/login
JWT, búsqueda de estaciones, endpoint de conexiones
- Sprint 2 (HU-08 a HU-14): historial de búsquedas, CRUD de
rutas y estaciones favoritas, stationboard con atajo de
favoritos, conexiones multi-parada, documentación Swagger
```

Nota

Poda este archivo con regularidad: si un error ya no puede repetirse, sácalo.

04 · CAPACIDADES

Skills.md

Define las capacidades reutilizables del agente.

Para qué sirve

Procedimientos que el agente puede ejecutar de forma consistente cada vez que se le pide: revisar un pull request, preparar una release, escribir tests o documentar una nueva funcionalidad. Convierte tareas repetitivas en procesos estandarizados.

Qué documentar aquí

- Cómo revisar un pull request en este proyecto
- Cómo preparar y etiquetar una release
- Cómo escribir tests para un nuevo endpoint
- Cómo documentar una funcionalidad nueva

Ejemplo real — proyecto SwissRoute

```
# Skills.md — SwissRoute

## Revisar un pull request
1. Confirmar que todo cambio en /controller tiene su DTO
2. Verificar que las llamadas WebClient nuevas tienen .onStatus()
3. Correr `mvn test` y revisar cobertura del módulo afectado
4. Revisar que no se expongan entidades JPA en las respuestas
5. Confirmar que el título sigue Conventional Commits

## Preparar una release
1. Verificar que todas las HU del sprint están cerradas
2. Correr la suite completa: `mvn clean verify`
3. Actualizar CHANGELOG.md con las HU incluidas
4. Etiquetar: git tag sprint-N -m 'Sprint N: <resumen>'
5. Confirmar que Swagger refleja los endpoints actuales

## Escribir tests para un endpoint nuevo
1. Test unitario del service con Mockito
2. Test del cliente WebClient con MockWebServer
(incluir caso de error 4xx/5xx del API externo)
3. Test de integración del controller con @WebMvcTest
4. Cubrir el caso feliz y al menos un caso de fallo del
API externo (timeout, 404, 429)

## Documentar una funcionalidad nueva
1. Anotar el endpoint con @Operation y @ApiResponse (Swagger)
2. Describir el DTO de entrada/salida en el schema
3. Añadir un ejemplo de request/response en Context.md
si el endpoint es estructuralmente relevante
```

Nota

Cada skill debe ser un procedimiento repetible, no una descripción general de la tarea.

05 · EQUIVALENCIAS

Cómo se llaman en otras herramientas

Los nombres de estos archivos cambian según la herramienta, pero la lógica de las cuatro capas se mantiene. Ejemplo en Hermes:

Esta plantilla	Hermes	Función
`Agents.md`	`SOUL.md`	Quién es el agente
`Context.md`	`AGENTS.md`	Instrucciones específicas del proyecto
`Memory.md`	`MEMORY.md`	Lo que el agente ha aprendido
—	`USER.md`	Quién eres tú

Cómo adaptar esta plantilla a tu propio proyecto

- Copia las cuatro secciones de ejemplo y reemplaza el contenido por el de tu proyecto.
- No dupliques información entre archivos: cada dato vive en una sola capa.
- Memory.md se poda con regularidad — Context.md casi nunca cambia de fondo.
- Skills.md crece con el proyecto: añade un procedimiento nuevo cada vez que repitas una tarea manualmente.

Plantilla creada a partir del reel “La estructura de 4 archivos para tu agente de IA” — @chanti.dev